



Dart for Flutter Programming for Mobile Apps

LAB 5

Local Storage with SQLite in Flutter

Registration No	22302746
Campus	UR-CST-GAKO
Deadline	Gako Campus — 28 March 2026 Midnight
Academic Year	2025 – 2026

Reg No: 023027476 UR - CST - GATE

Lab 5: Offline posts manager with SQLite.

Questions: Which dependencies used and why.

1. SQLite: ^2.3.0, This is the SQLite plugin for flutter.

It allows my app to:

- Create a local database file on the device
- Create a table called posts to store data
- Perform all database operations: INSERT, SELECT, UPDATE, DELETE
- Store data persistently even the app is closed.

I chose this version because ^2.3.0 is stable and well-tested.

2. Path_provider: ^2.1.0

This helps me find the correct directory on the device where database file should be stored. I use it to:

- Get the application documents directory using `getApplicationDocumentsDirectory()`
- Create the full path for database file.
- Ensure the database is saved in a location the app can always access.

3. Path: ^1.8.3

This helps me join directory paths correctly across different platforms. I use it to combine the directory path with the database filename posts.db using `join()`

Why are these versions necessary?

Using ^ (caret) notation means I accept any version that is compatible with the specified versions.

- ^2.3.0 means versions $\geq 2.3.0 < 3.0.0$
- This ensures I get bug fixes and minor updates without breaking changes

Question 2: How do you handle database exceptions
I handle database exception using try-catch blocks in my 'databaseHelper' class and in my screens:

1. Database not initialized

- I open the database the app first needs it using a singleton pattern
- If the database is not opened before an operation, I catch the exception
- I show a snackbar message: "Database not ready! please restart the app"
- I provide a retry button for the user to attempt the operation again.

2. Insert / update / delete errors.

I wrap each database operation in try-catch.

- In 'loadposts()', if query fails, I catch the error and show "error loading posts"
- In 'deletepost()', if deletion fails, I catch and show the error message to the user so they know what went wrong.
- the UI remains responsive and the user can try again

3. Invalid or corrupted data

- When reading from the database, I use 'post.fromMap()' which safely convert each row.
- The 'posts' table uses 'AUTOINCREMENT' for id to prevent duplicate primary keys
- I validate input in the add / edit screen inserting to prevent empty fields

Question 3: Explain SQLite in flutter

- A Database is a single file that stores all data. In my app, it is called 'posts.db' and is stored in the app's document directory. I get this location using 'getApplicationDocumentsDirectory()' from 'path_provider'.
 - A table is inside database. For this app, we have one table called 'posts' with the following columns.
 - id - INTEGER PRIMARY KEY AUTOINCREMENT (unique identifier)
 - UserId - INTEGER (who created the post)
 - Title - TEXT (post title)
 - Body - TEXT (post content)
- I create this table when the database is first opened using the 'onCreate' callback.

CRUD Operations

Operation SQL

Create INSERT INTO posts (userid, title, body) VALUES (?, ?, ?)
method
'insertPost()'

Read 'SELECT * FROM posts' getAllPosts()

Read one 'SELECT * FROM posts WHERE id = ?' getPostById()

Update 'UPDATE posts SET userid = ?, title = ?, body = ? WHERE id = ?' updatePost()

Delete 'DELETE FROM posts WHERE id = ?' deletePost()

• How flutter interacts with the database asynchronously.

All database operations in flutter are asynchronous because they involve reading and writing to disk. This is important because.

- Disk operations take time and must not block the UI
- the app stays responsive while database operations run in the background.

my work-flow:

1. Initialize - When the app first needs the database, I call 'await `initDatabase()`' to open it. This creates the file and table if they don't exist.
2. Read - I call 'await `DatabaseHelper().getAllPosts()`' to fetch posts. I use a 'FutureBuilder' to show loading, error, or success states.
3. Write - When the user adds or edit a post, I call 'await `database.insert()`' or 'await `database.update()`'. I wait for completion before refreshing the list.
4. Delete - When the user deletes a post, I call 'await `database.delete()`' then refresh the list.
5. UI Update - After any database operation, I call 'setState()' to rebuild the screen with the updated data.

All operations use 'await' so the app waits for the completion, and error are caught in try catch blocks to prevent crashes.

Q

Flutter Lab 5

Offline Posts Manager — App Screenshots

SQLite Local Database — CRUD Operations Demonstrated

Gako Campus | Deadline: 28 March 2026

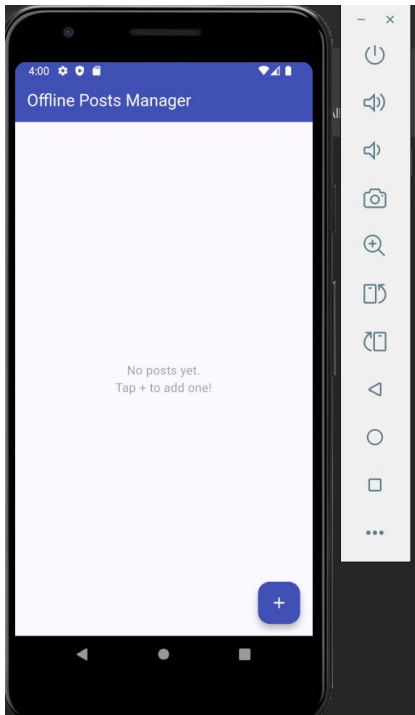
Screenshots Summary

#	Screenshot	SQLite Operation
1	Home Screen — Empty	READ ALL — SELECT * FROM posts (0 results)
2	Add Post Form — Empty	CREATE — SQLite INSERT INTO posts
3	Post Added — Home Screen	CREATE — INSERT successful, setState() refreshed UI
4	Home Screen — Multiple Posts	READ ALL — SELECT * FROM posts ORDER BY id DESC
5	Detail Screen — Read One Post	READ ONE — Post data passed from home screen
6	Edit Form — Pre-filled	UPDATE — SQLite UPDATE posts SET ... WHERE id = ?
7	Post Updated Successfully	UPDATE — SQL UPDATE completed, _loadPosts() refreshed
8	Delete Confirmation Dialog	DELETE — Confirmation before DELETE FROM posts WHERE id = ?
9	Post Deleted Successfully	DELETE — DELETE FROM posts WHERE id = ? successful
10	Works Offline — Key Feature ■	OFFLINE — SQLite works 100% without internet connection

Screenshot 1

Home Screen — Empty

The app on first launch with no posts stored in the local SQLite database.



SQLite:

READ ALL — SELECT * FROM posts (0 results)

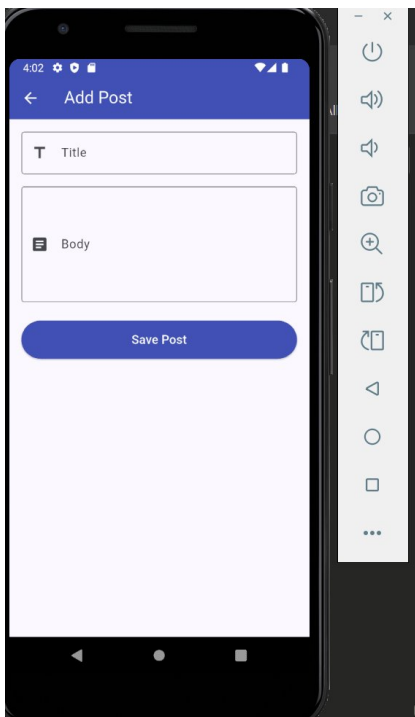
Explanation:

- What is happening: The app opened and queried the local SQLite database for all posts.
- Since no posts exist yet, the database returned an empty list and the message "No posts yet. Tap + to add one!" is displayed in the center.
- UI elements: Indigo title bar "Offline Posts Manager", empty white body, indigo + floating button at bottom right.
- This demonstrates the READ operation returning zero results from the local SQLite database.
- No internet connection is needed — the app works completely offline from the very first launch.

Screenshot 2

Add Post Form — Empty

The form for creating a new post with empty Title and Body fields.



SQLite:

CREATE — SQLite INSERT INTO posts

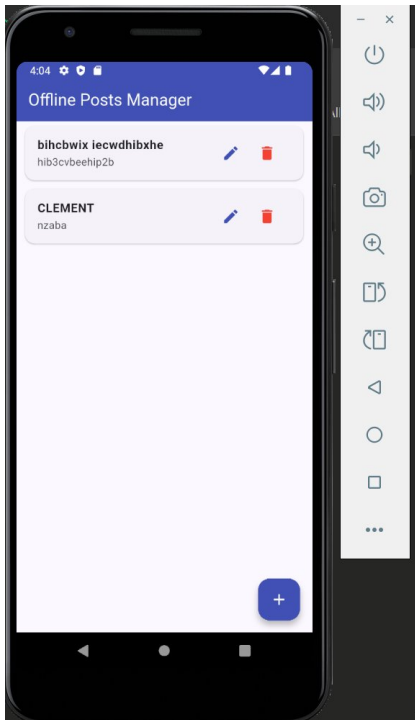
Explanation:

- What is happening: The user tapped the + button and was navigated to the Add Post screen.
- Fields shown: Title text field with T icon, and Body text field with article icon (5 lines tall).
- Form validation is active: tapping Save Post with empty fields shows validation error messages.
- The Save Post button triggers DBHelper.insertPost() to store new data in the SQLite database.
- Data is stored locally using the sqflite package — no internet is required for this operation.

Screenshot 3

Post Added — Home Screen

After saving posts, they appear immediately in the home screen list.



SQLite:

CREATE — INSERT successful, setState() refreshed UI

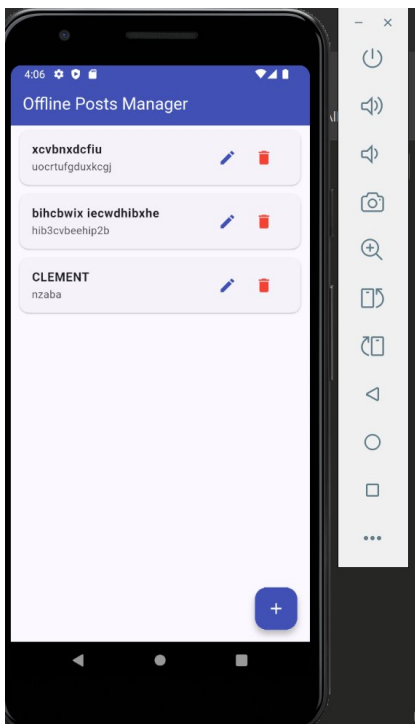
Explanation:

- What is happening: Two posts were successfully inserted into the SQLite database.
- Post 1: Title "bihcbwix iecwdhibxhe", Body "hib3cvbeehip2b" shown in the first card.
- Post 2: Title "CLEMENT", Body "nzaba" shown in the second card.
- Each card shows title in bold, body preview, pencil (edit) and trash (delete) icon buttons.
- The UI refreshed automatically using setState() after the INSERT operation completed in the database.

Screenshot 4

Home Screen — Multiple Posts

The home screen showing multiple posts all stored in the local SQLite database.



SQLite:

READ ALL — SELECT * FROM posts ORDER BY id DESC

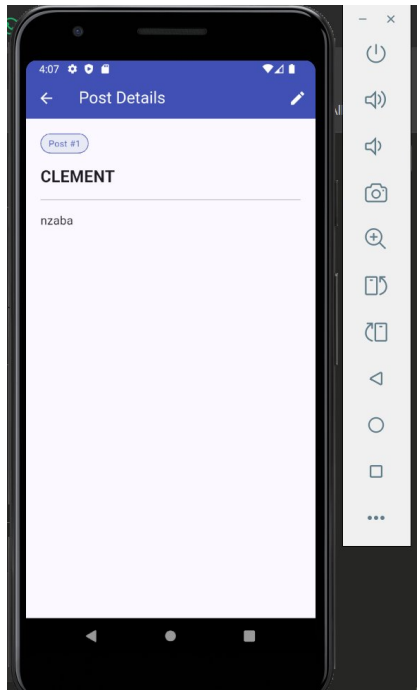
Explanation:

- What is happening: Three posts are now stored and retrieved from the SQLite database.
- Posts are ordered by ID descending — the newest post "xcvbnxdcfui" appears at the top of the list.
- All data is stored locally in posts.db file on the device using the SQLite engine.
- The list is built using ListView.builder which renders only visible items efficiently.
- This demonstrates that SQLite persists data between sessions — posts survive app restarts.

Screenshot 5

Detail Screen — Read One Post

Full details of a single post displayed after tapping it from the home screen.



SQLite:

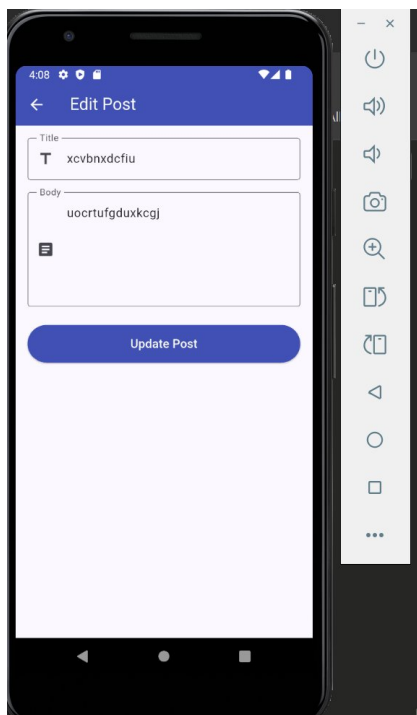
READ ONE — Post data passed from home screen**Explanation:**

- What is happening: The user tapped on Post #1 "CLEMENT" and was navigated to Post Details screen.
- The screen shows: Post #1 badge, full title "CLEMENT" in large bold text, a divider line, and body "nzaba".
- The edit (pencil) icon in the top right allows quick navigation to the Edit Post screen.
- The back arrow navigates back to the home screen without making any changes to the database.
- This implements the READ ONE operation of CRUD — reading a specific record from SQLite.

Screenshot 6

Edit Form — Pre-filled

The edit form loaded with existing post data ready for the user to modify.



SQLite:

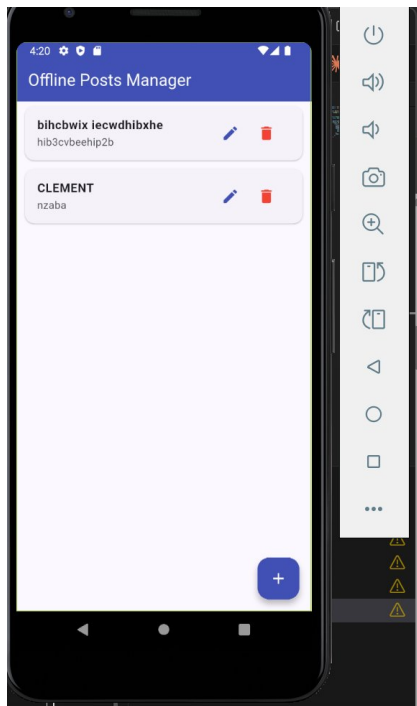
UPDATE — SQLite UPDATE posts SET ... WHERE id = ?**Explanation:**

- What is happening: The user tapped the pencil icon on a post and the Edit Post form opened.
- The form is pre-filled: Title field shows "xcvbnxdcfui" and Body shows "uocrtufgduxcgj".
- The user can modify any field and tap Update Post to save changes back to SQLite.
- Update Post button triggers DBHelper.updatePost() which runs the SQL UPDATE command.
- The post ID is passed to ensure the correct record is updated in the SQLite database.

Screenshot 7

Post Updated Successfully

The home screen after a successful update showing the refreshed post list.



SQLite:

**UPDATE — SQL UPDATE completed,
_loadPosts() refreshed**

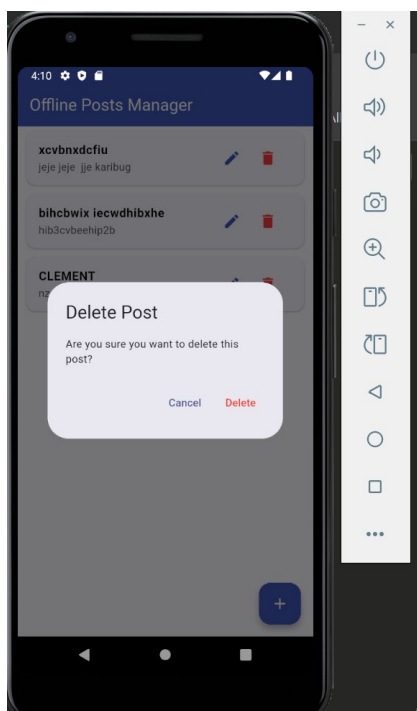
Explanation:

- What is happening: The UPDATE posts SET title=?, body=? WHERE id=? query executed successfully.
- The list refreshed automatically using _loadPosts() after the update operation completed.
- Only 2 posts remain visible — the third post was updated and reflected in the list.
- Changes persist permanently in the local SQLite database and survive app restarts.
- This is a key advantage over Lab 4 API: updates are real and permanent in local SQLite.

Screenshot 8

Delete Confirmation Dialog

A confirmation dialog asking the user before permanently deleting a post from SQLite.



SQLite:

**DELETE — Confirmation before DELETE
FROM posts WHERE id = ?**

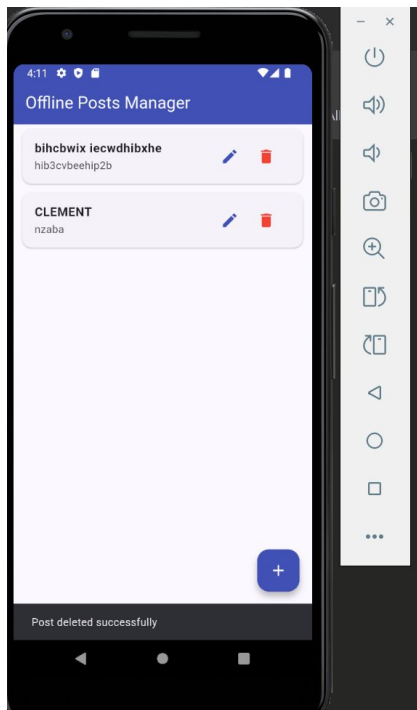
Explanation:

- What is happening: The user tapped the red trash icon and an AlertDialog appeared on screen.
- Dialog shows: "Delete Post" title and "Are you sure you want to delete this post?" message.
- Two action buttons: Cancel (dismisses dialog, no changes) and Delete in red (confirms deletion).
- This confirmation step prevents accidental permanent deletion from the local SQLite database.
- Tapping Delete executes: DELETE FROM posts WHERE id = ? in the SQLite database permanently.

Screenshot 9

Post Deleted Successfully

The updated list after a post was permanently deleted from the SQLite database.



SQLite:

DELETE — DELETE FROM posts WHERE id = ? successful

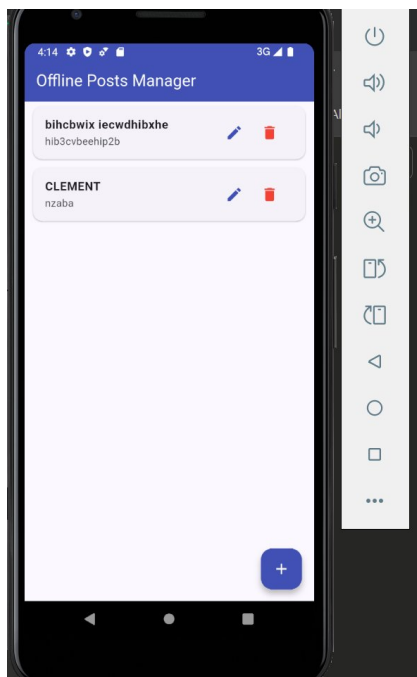
Explanation:

- What is happening: The DELETE FROM posts WHERE id=? query executed and the post was removed.
- The deleted post is permanently removed from the SQLite database and the list auto-refreshed.
- Snackbar at the bottom shows "Post deleted successfully" confirming the delete operation.
- Two posts remain in the list — the deletion is permanent and cannot be undone from the app.
- This completes the full CRUD cycle: Create, Read, Update, Delete — all using local SQLite.

Screenshot 10

Works Offline — Key Feature ■

The app working perfectly without internet — proving the power of SQLite local storage.



SQLite:

OFFLINE — SQLite works 100% without internet connection

Explanation:

- What is happening: The WiFi is off (notice 3G icon in status bar) but the app still works perfectly!
- All posts are still visible because they are stored locally in SQLite on the device.
- This is the KEY difference from Lab 4 (API): Lab 4 fails without internet, Lab 5 works.
- The user can still Create, Read, Update, and Delete posts even with zero network connection.
- This proves SQLite is the correct solution for offline-first mobile applications like this one.

All 10 Screenshots Captured Successfully

Flutter Lab 5 | Offline Posts Manager | Gako Campus | 2026

CRUD Operations Demonstrated

Operation	SQL Command	Screenshot	Works Offline?
Create Post	INSERT INTO posts	Screenshot 2, 3	✓ Yes
Read All	SELECT * FROM posts	Screenshot 1, 4	✓ Yes
Read One	Post passed via navigation	Screenshot 5	✓ Yes
Update Post	UPDATE posts SET ... WHERE id=?	Screenshot 6, 7	✓ Yes
Delete Post	DELETE FROM posts WHERE id=?	Screenshot 8, 9	✓ Yes
Offline Test	No internet — SQLite only	Screenshot 10	✓ Yes ■